Ombygging: totalvurderingen sendes med spillet

Fra ett API-kall per terning til ett kall for hele spillhylla. DiceRating blir en ren visningskomponent, og @ref-koblingen på spillsiden forsvinner

Instruksen viser hvordan totalvurderingen flyttes inn i `GameDTO`, slik at den følger med når spillene hentes. Den bygger på koden etter ombyggingen av POST og PUT. Hvert steg forklarer hva som endres og hvorfor.

Problemet: ett kall per spill

Før ombyggingen hentet hver `DiceRating` sin egen totalvurdering i `OnInitializedAsync`. Spillhylla henter først alle spillene i ett kall. Deretter lager den ett `GameCard` per spill, og hvert kort har en `DiceRating` som gjør sitt eget kall. Med åtte spill blir det ni kall for å vise én side.

```
Kallene på spillhylla

FØR: 1 + N kall
Games        → GET /api/games                          (alle spillene, uten total)
DiceRating 1 → GET /api/games/1/totalRatingByGame
DiceRating 2 → GET /api/games/2/totalRatingByGame
   ...
DiceRating 8 → GET /api/games/8/totalRatingByGame     = 9 kall

ETTER: 1 kall
Games        → GET /api/games                          (alle spillene, med total)
DiceRating   → viser BoardGame.TotalRating              = 1 kall
```

Mønsteret har et eget navn: **N+1-problemet**. Ett kall henter en liste, og så gjøres det ett nytt kall for hvert element i lista. Antall kall vokser med lista. Med 100 spill blir det 101 kall. Løsningen er å sende med det som trengs i det første kallet.

Problemet finnes også i designet. `DiceRating` har to ansvar: den viser terningkast, og den henter data fra API-et. På spillsiden måtte forelderen derfor få tak i komponenten med `@ref` og be den hente totalen på nytt. Det bryter med «data ned, hendelser opp», fordi forelderen styrer barnet ved å kalle en metode i det, i stedet for å gi det data.

Løsningen

- Serveren regner ut totalen og legger den i `GameDTO`.
- Klienten legger totalen i `BoardGame.TotalRating`.
- `DiceRating` viser `BoardGame.TotalRating`, som den allerede får som parameter, og gjør ingen kall selv.
- Spillsiden henter spillet på nytt etter at en vurdering er lagret, og oppdaterer `game.TotalRating`. `DiceRating` viser den nye verdien fordi forelderen tegnes på nytt.

Fil	Endring
`Models/BoardGame.cs`	Ny egenskap `TotalRating`
`Client/GameDTO.cs`	Nytt felt `TotalRating`
`ServerServices/ServerGamesService.cs`	`ToDto` henter totalen og blir `async`
`Client/GameApiClient.cs`	`ToBoardGame` kopierer totalen
`Components/Shared/DiceRating.razor`	Henting fjernes. Viser `BoardGame.TotalRating`

Fil	Endring
`Components/Pages/GamesDetails.razor`	`@ref` fjernes. Henter spillet på nytt etter lagring
`Components/Shared/GameCard.razor`	Ingen endring

> **Om kopiering fra PDF:** Lange linjer kan bli brutt når kode kopieres fra en PDF. Sjekk at hver linje står samlet etter innliming, særlig kommentarlinjer.

Steg 1: BoardGame får plass til totalen

`BoardGame` er modellen sidene bruker. Den trenger en egenskap for totalen, slik at `DiceRating` kan lese den fra spillet den allerede får. Totalen er et snitt og derfor `double`. Den er ikke en `int`, som `Rating`.

Models/BoardGame.cs: legg til under RatingCounter

```
public int RatingCounter { get; set; }

// Average of all ratings. Calculated by the server and sent with the game
public double TotalRating { get; set; }
```

Steg 2: GameDTO får feltet

`GameDTO` er det som sendes over nettverket som JSON. Feltet legges til sist. Rekkefølgen i en `record` med konstruktør betyr noe, og alle steder som lager en `GameDTO` må sende med verdien. Det eneste stedet er `ToDto` i neste steg.

Client/GameDTO.cs

```
namespace AK2.Client
{
    public sealed record GameDTO(
        int Id,
        string Name,
        string Category,
        int MinPlayers,
        int MaxPlayers,
        int Rating,
        string Description,
        double TotalRating
        );


}
```

Steg 3: Servicen regner ut totalen

Totalen regnes ut av lagringen med `GetTotalRatingByGame`, som allerede fantes. Fordi den metoden er `async`, må også `ToDto` bli `async` og returnere `Task<GameDTO>`. Før var `ToDto` `static`. Nå bruker den feltet `query`, så `static` må fjernes.

ServerServices/ServerGamesService.cs: bytt ut ToDto

```
// The total rating is calculated by the storage and sent with the game,
// so the client does not need a separate call per game to get it
private async Task<GameDTO> ToDto(BoardGame game) => new(
    game.Id,
    game.Name,
    game.Category,
    game.MinPlayers,
    game.MaxPlayers,
    game.Rating,
    game.Description,
    await query.GetTotalRatingByGame(game.Id));
```

`GetAllGamesAsync` brukte `games.Select(ToDto)`. `Select` kan ikke vente på en `async`-metode, så den byttes ut med en vanlig løkke som venter på hvert spill:

ServerServices/ServerGamesService.cs: bytt ut GetAllGamesAsync

```
public async Task<IReadOnlyList<GameDTO>> GetAllGamesAsync()
{
    var games = await query.GetAllGamesAsync();

    var dtos = new List<GameDTO>();
    foreach (var game in games)
    {
        dtos.Add(await ToDto(game));
    }
    return dtos;
}
```

De tre andre stedene som kaller `ToDto`, får `await` foran. Uten `await` ville metoden returnert en `Task` i stedet for en `GameDTO`, og det gir kompileringsfeil:

ServerServices/ServerGamesService.cs: legg til await

```
// CreateGameAsync
return await ToDto(game);

// UpdateGameAsync
return game is null ? null : await ToDto(game);

// GetGameByIdAsync
return game is null ? null : await ToDto(game);
```

> **Er dette fortsatt N+1?** Totalen hentes fortsatt én gang per spill, men nå inne på serveren, fra en liste i minnet. Det koster nesten ingenting. Det dyre var HTTP-kallene fra klienten, og de er borte. Med en database ville man latt databasen regne ut alle snittene i én spørring.

Steg 4: Klienten tar vare på totalen

`ToBoardGame` gjør om `GameDTO` til `BoardGame` på klientsiden. Uten denne linjen ville totalen kommet fram over nettverket, men blitt borte på veien inn i `BoardGame`.

Client/GameApiClient.cs

```
private static BoardGame ToBoardGame(GameDTO dto) =>
    new(dto.Name, dto.MinPlayers, dto.MaxPlayers, dto.Category)
    {
        Id = dto.Id,
        Rating = dto.Rating,
        Description = dto.Description,
        TotalRating = dto.TotalRating
    };
```

Steg 5: DiceRating slutter å hente data

Dette er steget som fjerner de ekstra kallene. `DiceRating` får allerede `BoardGame` som parameter, så den kan lese totalen derfra. Det som fjernes:

- `@inject GameApiClient ApiClient`, siden komponenten ikke lenger kaller API-et
- feltene `totalRating` og `error`, og feilmeldingen i markupen
- `OnInitializedAsync` og `RefreshTotalAsync`

Linjen som viser totalen, leser nå fra parameteren:

Components/Shared/DiceRating.razor

```
<p>Totalvurdering av spillet: @BoardGame?.TotalRating.ToString("0.0")</p>
```

Hele filen etter endringen. Emojiene i `<h3>` og i teksten om terningen kan ikke vises i PDF-en, så de er utelatt her. Behold dem fra din egen fil.

Components/Shared/DiceRating.razor

```
<h3>Terningkast</h3>

<p>Totalvurdering av spillet: @BoardGame?.TotalRating.ToString("0.0")</p>

<p>Din vurdering: @Value</p>

@if (ReadOnly == false)
{
    <p>Terning skal trilles for spillet</p>

    <div>
        <label>Ditt terningkast: <input type="number" min="1" max="6" @bind="newValue" /></label>
        <button type="button" @onclick="SaveAsync">Lagre vurdering</button>
    </div>

    @if (validationMessage is not null)
    {
        <p style="color:red">@validationMessage</p>
    }
}



@code {
    [Parameter] public BoardGame? BoardGame { get; set; }
    [Parameter] public int Value { get; set; }
    [Parameter] public EventCallback<int> ValueChanged { get; set; }
    [Parameter] public bool ReadOnly { get; set; } = true;

    // Local copy, so the component never changes its own Value parameter
    private int newValue;
    private string? validationMessage;

    protected override void OnParametersSet() => newValue = Value;

    private async Task SaveAsync()
    {
        if (newValue is < 1 or > 6)
        {
            validationMessage = "Terningkastet må være fra 1 til 6";
            return;
        }

        validationMessage = null;

        // Tells the parent about the new value
        await ValueChanged.InvokeAsync(newValue);
    }


}
```

`DiceRating` er nå en ren visningskomponent. Den får data ned (`BoardGame`, `Value` og `ReadOnly`) og sier fra opp (`ValueChanged`). Den vet ikke at det finnes et API. Det gjør den lettere å gjenbruke og å teste, for eksempel med bUnit, fordi testen ikke trenger en `GameApiClient`.

Steg 6: Spillsiden oppdaterer totalen selv

Før ba spillsiden `DiceRating` om å hente totalen på nytt via `@ref`. Nå er det spillsiden som eier dataene. Etter at vurderingen er lagret, henter den spillet på nytt og oppdaterer `game.TotalRating`. Når forelderen tegnes på nytt etter hendelsen, får `DiceRating` det samme `BoardGame`-objektet med ny total, og viser den.

Fjern `@ref` fra `DiceRating`:

```
Components/Pages/GamesDetails.razor

<DiceRating BoardGame="game" @key="game.Id" ReadOnly="false"
                @bind-Value="game.Rating" @bind-Value:after="UpdateRating" />
```

Slett feltet som holdt på referansen:

```
Components/Pages/GamesDetails.razor: slett

// The DiceRating component rendered on this page
private DiceRating? diceRating;
```

Bytt ut `UpdateRating`:

```
Components/Pages/GamesDetails.razor

private async Task UpdateRating()
{
    try
    {
        await ApiClient.UpdateRatingAsync(Id, new UpdateRatingRequest { Rating = game!.Rating });

        // Fetches the game again to get the new total. DiceRating shows it through the BoardGame parameter
        var updatedGame = await ApiClient.GetGameByIdAsync(Id);
        if (updatedGame is not null)
        {
            game.TotalRating = updatedGame.TotalRating;
        }
    }
    catch
    {
        error = "Oppdatering av vurdering feilet";
    }
}
```

Bare `TotalRating` kopieres over, ikke hele spillet. Da beholder `game` det samme objektet, og ingenting annet på siden endres.

Steg 7: GameCard

Ingen endring. `GameCard` sender allerede `BoardGame="BoardGame"` til `DiceRating`, og `BoardGame` har nå totalen med seg.

Fordelen med endringen

- **Færre kall.** Spillhylla går fra 1 + N kall til ett. Med åtte spill blir det ett kall i stedet for ni, og antall kall vokser ikke lenger med samlingen.
- **Raskere side.** Kortene viser totalen med en gang. Før kom totalene inn én etter én, etter hvert som kallene ble ferdige.
- **Tydeligere ansvar.** Servicen setter sammen dataene, og `DiceRating` viser dem. Komponenten har ett ansvar i stedet for to.
- **«Data ned, hendelser opp».** Spillsiden eier totalen og gir den til `DiceRating` via parameteren. Forelderen trenger ikke lenger å styre barnet med `@ref`.
- **Enklere å teste.** `DiceRating` trenger ingen `GameApiClient`, så en bUnit-test kan bare sende inn et `BoardGame`.

> **Hva som ikke blir bedre:** Spillsiden gjør fortsatt to kall når en vurdering lagres, PUT og deretter GET. Det var det samme før. Skal det ned til ett kall, må `PUT /api/games/{id}/rating` svare med den nye totalen, på samme måte som POST og PUT svarer med spillet.

Testet mot API-et

Kall	Svar
`GET /api/games`	Alle spillene med `"totalRating": 0`
`PUT /api/games/1/rating` med 6, og så med 3	`204` begge gangene
`GET /api/games/1`	`"rating": 3, "totalRating": 4.5`
`POST /api/games` og `PUT /api/games/9`	Svarene har `totalRating` med

Ikke testet her: sidene i nettleseren. Koden kompilerer. Sjekklisten under er første gang sidene kjøres.

Sjekk i nettleseren	Forventet
Åpne spillhylla	Alle kortene viser totalvurdering med en gang
Se i Output-vinduet i Visual Studio	Ett `GET /api/games`, ingen `totalRatingByGame`-kall
Gi Monopol 6 og deretter 3 på spillsiden	Totalen viser 6,0 og deretter 4,5
Gå tilbake til spillhylla	Monopol-kortet viser 4,5
«Neste spill» på spillsiden	Totalen til det nye spillet vises

Det som står igjen

- `GetTotalRatingByGame` **i** `GameApiClient` og endepunktet `GET /api/games/{id}/totalRatingByGame` brukes ikke lenger av sidene. Om de skal fjernes eller beholdes for andre klienter, er et eget valg. `GetTotalRatingByGame` i servicen og lagringen må uansett beholdes, fordi `ToDto` bruker den.
- **README-en:** Står det noe om `@ref` og `RefreshTotalAsync` der, stemmer det ikke lenger med koden.
- **Spillekvelden** lagrer `BoardGame` i nettleseren. Totalen blir med i det lagrede, men vises ikke på `/game-night`, så det har ingen betydning.